

MCP

Model Context Protocol

One standard way for AI models to use tools and data

Write a connection once, use it from any AI app

AI/ML Engineer Learning Series · Deep Dive Edition

What it is	A standard protocol connecting AI to tools/data
Made by	Anthropic (open standard, anyone can use it)
The problem it fixes	Every model needing custom tool wiring
Core idea	One shared 'USB port' for AI tool connections
Builds on	Tools and agent concepts you already know

What's Inside

Here is everything covered in this guide, in order:

#	Section	What you'll learn
1	The Problem MCP Solves	Custom wiring for every tool
2	What is MCP?	One shared protocol
3	Client and Server	The two sides
4	What a Server Exposes	Tools, resources, prompts
5	Build Once, Use Anywhere	The portability win
6	A Simple MCP Server	Real code
7	How a Client Uses It	Connecting and calling
8	MCP vs Plain Function Calling	What's actually new
9	Common Mistakes	What to avoid
★	Cheatsheet	Quick reference

1. The Problem MCP Solves

Remember tools from the Agentic AI and LangChain topics — functions an LLM can call to search, read files, or use an API. The problem: without a shared standard, every AI app has to write its OWN custom code to connect to every tool. Connect an app to Slack, Google Drive, and a database, and you write three separate, model-specific integrations. Do that again for a different AI model, and you write them all over again.

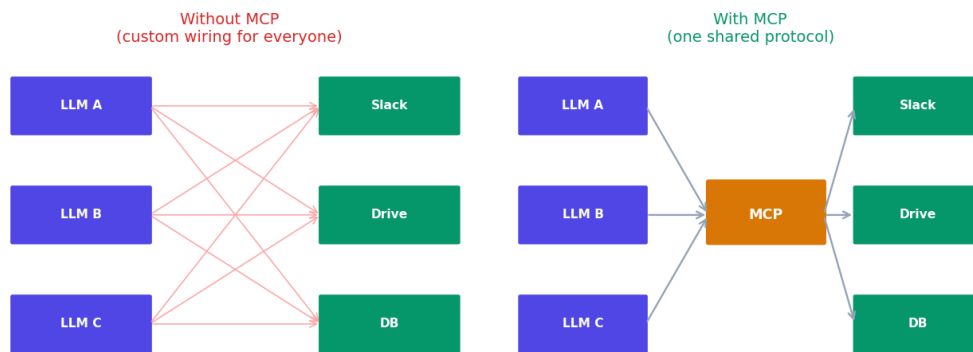


Fig 1 — Without a standard, every model needs custom wiring to every tool. MCP replaces that mess with one shared protocol.

This is the classic 'N models times M tools' problem — the number of custom integrations explodes as you add more models and more tools. MCP (Model Context Protocol) fixes this by giving everyone one shared, standard way to connect AI models to tools and data. Write the connection once, using MCP, and any MCP-compatible AI app can use it.

■ MCP was introduced by Anthropic as an open standard — free for anyone to use or build on, not locked to one company's models. It's quickly become a common way to connect AI apps (including Claude) to real tools and data.

2. What is MCP?

MCP stands for Model Context Protocol. A helpful comparison: it's like a USB port for AI. Before USB, every device needed its own special cable and port. USB gave everyone one standard plug that just works, everywhere. MCP does the same for connecting AI models to tools, files, and data — one standard 'plug' instead of custom wiring for every pair.

Practically, MCP defines a standard way for two programs to talk: an AI application (like Claude) on one side, and a 'server' that exposes some tool or data on the other. As long as both sides speak MCP, they can connect — no matter who built which side.

3. Client and Server

MCP has two sides, and the names matter. The MCP Client is the AI application — the one using tools (for example, Claude, or your own agent). The MCP Server is the thing being connected to — it exposes tools, data, or capabilities that the client can use.

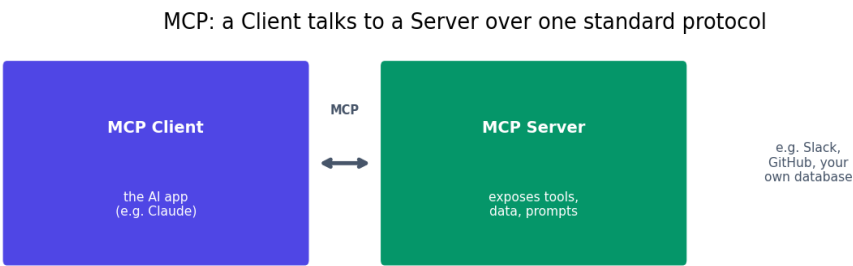


Fig 2 — An MCP Client (the AI app) talks to an MCP Server (which exposes tools and data) over the MCP protocol.

Role	What it does	Example
MCP Client	Uses tools and data via MCP	Claude, your custom agent app
MCP Server	Exposes tools and data via MCP	A Slack connector, a database connector

This should feel familiar — it's the same client-server idea from the FastAPI topic, applied specifically to connecting AI models with tools. The server does the real work (reading files, calling an API); the client is the AI deciding when and how to use it.

4. What a Server Exposes

An MCP server can offer three kinds of things to a client. Together, these cover almost everything an agent might need from an external system.

An MCP Server Can Expose Three Things

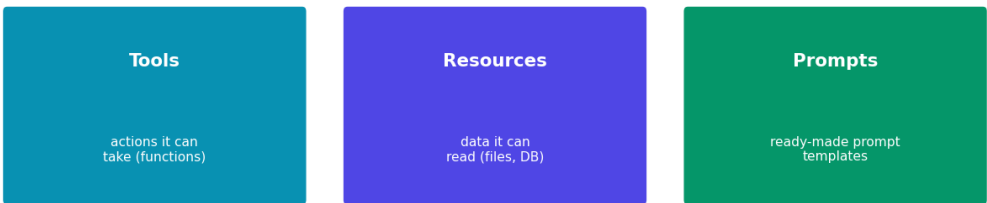


Fig 3 — An MCP server can expose Tools (actions), Resources (data), and Prompts (templates).

Type	What it is	Example
Tools	Actions the AI can take (like functions)	send_message(), search_files()
Resources	Data the AI can read	a file, a database table, a webpage
Prompts	Ready-made prompt templates	a template for summarising a doc

Tools are the most familiar — they map directly onto the 'tools' idea from Topic 36 and LangChain. Resources let a model read data without needing a special tool call for it. Prompts are reusable templates the server can offer, so common tasks are consistent every time they're used.

5. Build Once, Use Anywhere

This is MCP's biggest practical win. Because it's a shared standard, an MCP server you build once can be used by ANY MCP-compatible client — Claude, another AI app, or your own custom agent — without changing the server at all.

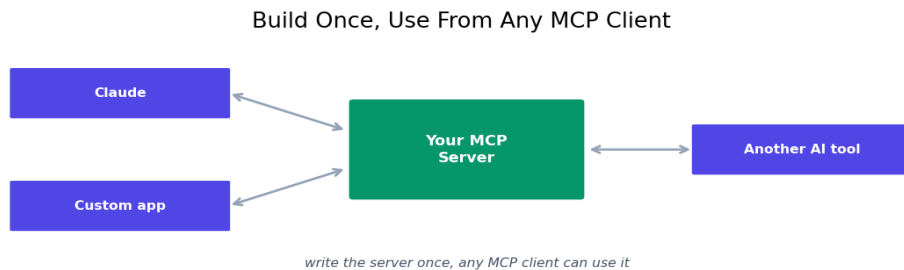


Fig 4 — Build your MCP server once; any MCP client (Claude, your own app, others) can connect to it.

Compare this to writing a custom tool integration baked into one specific app — useful only there. An MCP server is reusable infrastructure: connect your company's database once via MCP, and it works for every AI tool your team uses, today and in the future, without rebuilding the connection each time.

■ *This reusability is why MCP matters for real engineering work, not just for one chatbot. It's infrastructure you build once and get value from repeatedly — a good habit to bring into any AI project with external data or tools.*

6. A Simple MCP Server

Here's a minimal MCP server exposing one tool. The shape should feel familiar from the tools you've already written for agents — MCP just adds a standard wrapper around it.

```
# pip install mcp
from mcp.server.fastmcp import FastMCP

mcp = FastMCP('my-tools-server')

@mcp.tool()
def get_weather(city: str) -> str:
    '''Get the current weather for a city.'''
    return weather_api(city) # your real function

@mcp.tool()
def search_my_pdf(question: str) -> str:
    '''Search the user's documents (your RAG chatbot).'''
    return rag_retrieve(question)

if __name__ == '__main__':
    mcp.run()
```

Notice `search_my_pdf` — exactly your RAG chatbot, now wrapped as an MCP tool instead of a plain LangChain/agent tool. Once wrapped this way, ANY MCP client, including Claude, could use your RAG

chatbot as a tool, not just the one agent app you originally built it for.

7. How a Client Uses It

On the client side, connecting to an MCP server is mostly configuration — you tell the client where the server is, and it automatically discovers what tools, resources, and prompts are available.

```
# Example: connecting a Python MCP client to a server
from mcp import ClientSession
from mcp.client.stdio import stdio_client

async with stdio_client(server_params) as (read, write):
    async with ClientSession(read, write) as session:
        await session.initialize()

        # Discover what the server offers
        tools = await session.list_tools()

        # Call a tool, just like calling a function
        result = await session.call_tool(
            'get_weather', {'city': 'Dhaka'})
```

The client doesn't need to know how `get_weather` works internally — it just discovers the tool exists and calls it through the standard protocol. This discovery step is part of what makes MCP powerful: clients can adapt to whatever tools a server happens to offer, without hardcoding them in advance.

■ *In practice, many AI apps (including Claude's interfaces) let you connect MCP servers through simple settings, without writing this client code yourself. Knowing what happens underneath still helps you build and debug your own servers.*

8. MCP vs Plain Function Calling

You may wonder: isn't this just the 'tools' idea from Topic 36 and LangChain again? Mostly yes — MCP doesn't invent a new concept, it standardises HOW that concept connects across different apps.

	Plain function calling	MCP
Where it's defined	Inside one specific app	A separate, reusable server
Reuse across apps	Rebuild it each time	Connect any MCP client instantly
Discovery	You hardcode what's available	Client can discover tools automatically
Best for	A single, simple project	Shared infrastructure, multiple apps

Simple way to think about it: if you're building one tool for one agent in one project, plain function calling (like in Topic 36) is perfectly fine and simpler. Reach for MCP when you want that tool to be reusable infrastructure — usable from Claude, from your own apps, and from anything else that speaks MCP, without rebuilding the connection each time.

9. Common Mistakes

Mistake	Why it's a problem	Fix
Building MCP for a one-off tool	Unneeded complexity	Use plain function calling instead
No docstrings on tools	Client can't tell what a tool does	Write clear tool descriptions
Exposing risky tools by default	Unsafe actions run automatically	Gate sensitive tools behind confirmation
Ignoring resources vs tools	Misusing tools for plain data reads	Use Resources for read-only data
Not testing discovery	Client can't find your tools	Verify <code>list_tools()</code> shows everything
Treating it as a new AI concept	Confusion about what's new	Remember: it's tools, standardised

Quick Reference Cheatsheet

Concept / Task	Meaning or Code
MCP	standard protocol for AI-to-tool connections
Made by	Anthropic (open standard)
MCP Client	the AI app using tools (e.g. Claude)
MCP Server	exposes tools/data/prompts
Tools	actions the AI can take
Resources	data the AI can read
Prompts	ready-made prompt templates
Install	<code>pip install mcp</code>
Create server	<code>FastMCP('name')</code>
Define a tool	<code>@mcp.tool()</code> above a function
Run server	<code>mcp.run()</code>
Client discovers tools	<code>session.list_tools()</code>
Client calls a tool	<code>session.call_tool(name, args)</code>

The one idea to remember: MCP is a shared, standard way for AI models to connect to tools and data — like a USB port for AI. Build a server once, exposing tools, resources, and prompts, and any MCP-compatible client (including Claude) can use it, without custom wiring for every pair of model and tool.

This is a bonus topic, extending your AI Agents stage beyond the original 44. It builds directly on the tools concept from Agentic AI Concepts and LangChain — MCP is how that same idea becomes reusable, shareable infrastructure.